

R basic (I)

Function documentations

get help for the function

>help("mean")

>?mean

this will search all the installed packages including those not yet loaded

>help.search("mean")

>??mean

: everything after # in the line is remark.
i.e. R will ignore it.

Function documentations (cont'd)

get the functions with name including "keyword"

>apropos("mean")

[1] ".colMeans" ".rowMeans" "colMeans" "kmeans"

[5] "mean" "mean.Date" "mean.default" "mean.difftime"

[9] "mean.POSIXct" "mean.POSIXlt" "rowMeans" "weighted.mean"

demo function or example

>demo()

>demo(graphics)

>example(glm)

R 是計算機

```
> 2.4 + 42
```

```
[1] 44.4
```

```
> 4 ^ 2
```

```
[1] 16
```

```
> sqrt(100)
```

```
[1] 10
```

```
> 100 ^ 0.5
```

```
[1] 10
```

```
> a <- 1
```

```
> a
```

```
[1] 1
```

```
> 1 -> b
```

```
> b
```

```
[1] 1
```

```
> a + b
```

```
[1] 2
```

```
> m <- c(3, 6, 4)
```

```
> n = c(1, 2, 3)
```

```
> m + n
```

```
[1] 4 8 7
```

```
> m - n
```

```
[1] 2 4 1
```

```
> m * n
```

```
[1] 3 12 12
```

```
> m / n
```

```
[1] 3.000 3.000 1.333
```

資料屬性

R 的基本資料屬性包含以下五種，可用 `class` 函數判斷資料屬性

- `character`：文字字串，用 `"` 包起來，ex：`"test"`
- `numeric`：實數
- `integer`：整數
- `complex`：複數
- `logical`：True 或 False

```
> class("test")  
[1] "character"
```

```
> class(10.10)  
[1] "numeric"
```

```
> class(10)  
[1] "numeric"
```

```
> class(as.integer(3)) # 因為 R 計算上都是都是以雙倍精確度來計算，所以必須指定為 integer，不然都會被當成 numeric 看待。  
[1] "integer"
```

```
> class(as.integer(3.1)) # as.integer 可以將不是整數的數值變成整數  
[1] "integer"
```

```
> class(as.integer(T)) # as.integer(T) = 1  
[1] "integer"
```

```
> class(as.integer(F)) # as.integer(T) = 0  
[1] "integer"
```

```
> class(2+2i)  
[1] "complex"
```

```
> class(TRUE) # 注意都要大寫，不可寫 True，但可以簡化成 T  
[1] "logical"
```

```
> class(T)  
[1] "logical"
```

常見運算

加減乘除

```
> 1 + 2
```

```
[1] 3
```

```
> 1 - 2
```

```
[1] -1
```

```
> 1 * 2
```

```
[1] 2
```

```
> 1 / 2
```

```
[1] 0.5
```

次方、平方根、商數與餘數

```
> 2 ^ 3 # 2 的 3 次方
```

```
[1] 8
```

```
> 2 ** 3 # 2 的 3 次方
```

```
[1] 8
```

```
> sqrt(4) # 4 的平方根
```

```
[1] 2
```

```
> 27 ^ (1/3) # 27 的立方根
```

```
[1] 3
```

```
> 11 %/% 5 # 11 除以 5 的商數
```

```
[1] 2
```

```
> 11 %% 5 # 11 除以 5 的餘數
```

```
[1] 1
```

常見運算

sign：判斷是正、負數或 0

- 正數回傳 1
- 負數回傳 -1
- 0 回傳 0

```
> sign(10)
[1] 1

> sign(0)
[1] 0

> sign(-10)
[1] -1
```

abs：取絕對值

```
> abs(10)
[1] 10

> abs(0)
[1] 0

> abs(-10)
[1] 10
```


常見運算

log

```
> log(10) # log 以 e 為底
[1] 2.302585

> log1p(9) # log(x) = log1p(x - 1)
[1] 2.302585

> log(10, 2) # 指定 log 以 2 為底
[1] 3.321928

> log2(10) # log2 代表以 2 為底
[1] 3.321928

> log10(10) # log10 代表以 10 為底
[1] 1
```

exp

```
> exp(10)
[1] 22026.47

> expm1(10) # expm1(x) = exp(x) - 1
[1] 22025.47
```

變數與資料

1. 變數
2. 向量
3. 矩陣
4. 因子
5. 串列
6. 資料框架
7. 時間數列
8. 指標

變數

R 在給予變數值時是利用「<-」並不是程式語言中常見的「=」，在 [Google's R Style Guide](#) 與 [R 官方文件](#) 都強調不該使用「=」，因為在某些狀況是會失效的。另外在 R 變數命名上，大小寫是有區別的，所以 x 與 X 其實是不同的變數。

```
> x <- 1
> y <- 2
> x + y
[1] 3

> x1 <- x2 <- 1 # x1 與 x2 都是 1
> x1 + x2
[1] 2
```

R 的變數可以重複給予值，不會因為資料屬性的不同而發生錯誤，會因最後所給予的值為結果。所以程式碼複雜時，常常會因為一個變數重複給予不同的值而發生錯誤，這時可以用 `exists` 函數檢查。

```
> x = 1
> x
[1] 1
> x = 1.3
> x
[1] 1.3
> x = 1 + 2i
> x
[1] 1+2i
> x = "test"
> x
[1] "test"
> x = FALSE
> x
[1] FALSE

> x = 10
> exists("x")
[1] TRUE
```

NA 與 NULL

NA 代表是個空物件，已經有物件但是裡面沒東西，NULL 則是根本沒有任何東西，更詳細比較請參考 [R Bloggers R : NA vs. NULL](#)

Variable naming:

Letters and numbers are allowed (but the [first character has to be a letter](#)). “.” is also allowed.

向量 (vector)
利用 `c(...)` 建立
向量，但切記
向量元素必須
是同個資料 屬
性。

P.S.：經由以上比較後，可以得到當放入的
形態不同時，會被轉成同一形態，且可以每
個形態的強弱不同，以下是強到弱排序。

character > complex > numeric > integer > logical

```
> c(1, 2, 3)
```

```
[1] 1 2 3
```

```
> c(1, TRUE, "test") # 全部都變成 character
```

```
[1] "1"      "TRUE" "test"
```

```
> c(1.1, TRUE, "test") # 全部都變成 character
```

```
[1] "1.1"    "TRUE" "test"
```

```
> c(1+2i, TRUE, "test") # 全部都變成 character
```

```
[1] "1+2i"   "TRUE" "test"
```

```
> c(1, TRUE) # 全部自動轉成 integer
```

```
[1] 1 1
```

```
> c(1.1, TRUE) # 全部自動轉成 numeric
```

```
[1] 1.1 1.0
```

```
> c(1+2i, TRUE) # 全部自動轉成 complex
```

```
[1] 1+2i 1+0i
```

```
> c(1, 1.1) # 全部自動轉成 numeric
```

```
[1] 1.0 1.1
```

```
> c(1, 1.1, 1+2i) # 全部自動轉成 complex
```

```
[1] 1.0+0i 1.1+0i 1.0+2i
```

透過指標與名稱提取資料

我們可以透過以下二種方式取得向量元素。

- 指標
- 元素名稱

另外可以搭配 `[]` 或 `[[ ]]`，這樣分別會回傳向量元素的所有資訊或向量元素的數值，總共可以分成以下四種狀況。

- `x[i]`：回傳向量元素所有資訊
- `x[[i]]`：只回傳向量元素的值
- `x[元素名稱]`：回傳向量元素所有資訊
- `x[[元素名稱]]`：只回傳向量元素的值

```
> x <- c(joe=12, vicky=14, bob=17)
```

```
> x[1]
```

```
joe
```

```
12
```

```
> x[[1]]
```

```
[1] 12
```

```
> x["joe"]
```

```
joe
```

```
12
```

```
> x[["joe"]]
```

```
[1] 12
```

```
> x[1]
```

```
joe
```

```
12
```

```
> x[[1]]
```

```
[1] 12
```

```
> x[1:2] # 一次取多個向量元素
```

```
joe vicky
```

```
12    14
```

c(...) 類似的函數 x:y 、 seq 與 rep

- `x:y`：回傳 `x` 到 `y` 的整數向量，所以 `x` 與 `y` 都是整數。
- `seq(s, e, by)`：產生一個等差級數的向量。
 - `s` 是初始值
 - `e` 是結束值
 - `by` 是遞增值，預設是 1
- `rep(x, times, each)`：產生一個重覆循環的向量。
 - `x` 是需重覆循環的數值
 - `times` 是重覆循環次數
 - `each` 是 `x` 內元素重覆的次數

```
> 1:5
[1] 1 2 3 4 5

> seq(1, 5)
[1] 1 2 3 4 5

> seq(1, 5, 0.3) # 就算沒有剛好加到跟結束值一樣也沒關係
[1] 1.0 1.3 1.6 1.9 2.2 2.5 2.8 3.1 3.4 3.7 4.0 4.3 4.6 4.9

> rep(c(1, 2, 3), times = 3, each = 2)
[1] 1 1 2 2 3 3 1 1 2 2 3 3 1 1 2 2 3 3

> rep(1:4, times = 3, each = 2)
[1] 1 1 2 2 3 3 4 4 1 1 2 2 3 3 4 4 1 1 2 2 3 3 4 4
```

```
>x<-1:10
>x<-seq(-5, 5, by=.2)
>x<-seq(-5, 5, length=7)
>x<-rep(1:3, times=5) ## same with x<-rep(1:3, 5)
>x<-rep(1:3, each=5)
>x<-rep(1:3, length=5)
```

基本相關函數

- 向量加減乘除
- `length`：計算向量中的元素個數。
- `sum`：將向量所有元素加總。
- `prod`：將向量所有元素相乘。
- `cumsum`：回傳元素累加向量。
- `cumprod`：回傳元素累乘向量。
- `sort`：將向量元素排列，產生排序過的向量。
- `rank`：回傳各向量元素的排序值。

```
> c(3, 4, 2) + c(3, 4, 2)
[1] 6 8 4
> c(3, 4, 2) - c(3, 4, 2)
[1] 0 0 0
> c(3, 4, 2) * c(3, 4, 2)
[1] 9 16 4
> c(3, 4, 2) / c(3, 4, 2)
[1] 1 1 1
```

```
> length(c(3, 4, 2))
[1] 3
```

```
> sum(c(3, 4, 2))
[1] 9
```

```
> cumsum(c(3, 4, 2))
[1] 3 7 9
```

```
> cumprod(c(3, 4, 2))
[1] 3 12 24
```

```
> prod(c(3, 4, 2))
[1] 24
```

```
> sort(c(3, 4, 2))
[1] 2 3 4
```

```
> rank(c(3, 4, 2))
[1] 2 3 1
```

上述向量加減乘除時，向量個數都是一樣的狀況，但不同的狀況會發生什麼結果，如下：

如果長度較長的向量長度是較短的倍數的話是可以相加或相減的。

```
> c(1, 2) + c(1, 2, 3)
```

```
[1] 2 4 4
```

警告訊息：

```
In c(1, 2) + c(1, 2, 3) : 較長的物件長度並非較短物件長度的倍數
```

```
> c(1, 2) + c(1, 2, 3, 4) # 1+1 2+2 1+3 2+4
```

```
[1] 2 4 4 6
```

乘跟除

乘跟除的情況與加跟減狀況一致，只是當長度不是倍數時會有結果但也會警告。

```
> c(1, 2) * c(1, 2, 3) # 1*1 2*2 1*3
```

```
[1] 1 4 3
```

警告訊息：

```
In c(1, 2) * c(1, 2, 3) : 較長的物件長度並非較短物件長度的倍數
```

```
> c(1, 2) * c(1, 2, 3, 4) # 1*1 2*2 1*3 2*4
```

```
[1] 1 4 3 8
```

```
>x<-c(1, 2, NA, 3, 4) ## NA is missing value
```

```
>is.na(x)
```

```
>length(x)
```

the command won't run. The default parameter for mean doesn't allow missing value.

```
>mean(x)
```

set the parameter so the mean function allows missing value

```
>mean(x, na.rm=T)
```

```
>x<- c(10.4, 5.6, 3.1, 6.4, 21.7)
```

```
>var(x)
```

```
>sum( (x-mean(x))^2 ) / (length(x)-1)
```

```
>x<-numeric() ## make it an empty vector with length indefinite  
>x[3]<-7      ## now x has length 3 with the third element equals 7.
```

Random number generator:

```
## get 1000 samples from normal distribution, means 5 and standard  
deviation 1.
```

```
> a<-rnorm(1000, 5, 1)  
> hist(a, nclass=30)    # hist(a, breaks=30) ← pretty values  
> hist(a, breaks=seq(min(a), max(a), (max(a)-min(a))/30))  
> pnorm(1.96)
```

rnorm, runif, rt, rchisq, rexp, rbeta, rgamma

dnorm (density), pnorm (cumulative probability), qnorm (quantile)

- Logical

```
>vec<-1:9
```

```
> as.vector(vec,mode="logical")
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

```
> as.vector(rep(0,9),mode="logical")
```

```
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

- Integer

```
> as.vector(vec,mode="integer")
```

```
[1] 1 2 3 4 5 6 7 8 9
```

```
> rand<-rnorm(9,0.5,0.3)
```

```
> rand
```

```
[1] 0.7005204 0.5449772 0.4083230 0.2198921 0.6539840 0.2354353 0.8684642
```

```
[8] 0.3442125 0.1168064
```

```
> as.vector(rand,mode="integer")
```

```
[1] 0 0 0 0 0 0 0 0 0
```

- Character

```
> cha<-as.vector(vec,mode="character")
> cha
[1] "1" "2" "3" "4" "5" "6" "7" "8" "9"
> is.numeric(cha)
[1] FALSE
> cha+1
錯誤在cha + 1 : 二元運算子中有非數值引數
```

- Numeric

```
> num<-as.vector(cha,mode="numeric")
> num
[1] 1 2 3 4 5 6 7 8 9
> is.numeric(num)
[1] TRUE
> num+1
[1] 2 3 4 5 6 7 8 9 10
```

- Complex

```
> as.vector(vec,mode="complex")
[1] 1+0i 2+0i 3+0i 4+0i 5+0i 6+0i 7+0i 8+0i 9+0i
```

- **mode, str, fix**

>x<-1:5 ## mode: numeric, complex, logical, character, list, function

>mode(x)

>str(x)

>fix(x)

>x<-as.character(x)

>mode(x)

>str(x)

>fix(x)

- logical value

TRUE(T) & FALSE(F)

>x[1]<0

>x<- (-5)+(0:15)

>x>0

>x>0 & x<=5

- operators

- <,<=,>,>=,==,!=

>x ==3

- NA : missing value

- Inf : infinite

>-3/0

- NaN : not a number

>0/0

>Inf-Inf

>log(-2)

- NULL : clear content

> x<-NULL

- String

```
>a<-"g"
```

```
>a
```

```
>b<-c(1, "x",2)
```

```
>b
```

```
>x<-paste("Affy", 1:5, sep="")
```

```
>x<-paste(c("X", "Y"), 1:5, sep="_")
```

- Elements select

```
>x<- (-5)+(0:15)
```

```
>x[1]
```

```
>x[1:3]
```

```
>x[-(3:6)]
```

```
>x[x>0 & x<=5]
```

```
>x[-(x>0 & x<=5)]
```

```
>x[1:3]<-100:102
```

```
>x[(x>0 & x<=5)]<-999
```

流程控制-邏輯判斷式

control_statements.R

常見的邏輯判斷符號

以下介紹常見的邏輯判斷的符號。

- <、>：小於、大於。
- <=、>=：小於等於、大於等於。
- ==、!=：等於、不等於。
- A %in% B：A 是否在 B 中。
- &&、&：交集，& 適用於向量式的邏輯判斷，&& 適用於單一值的邏輯判斷。
- ||、|：聯集，| 適用狀況與 & 相同，|| 適用狀況與 && 相同。

流程控制-條件執行

control_statements.R

常見的條件執行

以下介紹三種常見的條件執行。

- if else
- if else if else
- switch

流程控制-迴圈結構

control_statements.R

以下介紹三種迴圈與兩種改變迴圈狀態的方法。

- for
- while
- repeat
- break
- next

陣列(array) 利用 rbind、cbind 與 array 函數建立陣列

陣列可視為多維度的向量變數，跟向量一樣，所有陣列元素的資料屬性必須一致。

```
> x <- c(1, 2, 3)
```

```
> y <- c(4, 5, 6)
```

```
> rbind(x, y) # rbind 是利用 row(橫) 合併。
```

```
  [,1] [,2] [,3]
```

```
x     1     2     3
```

```
y     4     5     6
```

```
> cbind(x, y) # cbind 是利用 column(直) 合併。
```

```
  x y
```

```
[1,] 1 4
```

```
[2,] 2 5
```

```
[3,] 3 6
```

```
> array(x,c(1,3)) # c(1,3) 代表產生 1 x 3 陣列
```

```
  [,1] [,2] [,3]
```

```
[1,]    1    2    3
```

```
> array(x,c(2,3)) # c(2,3) 代表產生 2 x 3 陣列
```

```
  [,1] [,2] [,3]
```

```
[1,]    1    3    2
```

```
[2,]    2    1    3
```

```
> array(x,c(3,3)) # c(3,3) 代表產生 3 x 3 陣列
```

```
  [,1] [,2] [,3]
```

```
[1,]    1    1    1
```

```
[2,]    2    2    2
```

```
[3,]    3    3    3
```

陣列(array) 透過指標提取資料

陣列與向量相同，可以透過指標或名稱選取陣列的元素。

```
> x <- c(1, 2, 3)
> y <- c(4, 5, 6)

> z = rbind(x, y)
> z
  [,1] [,2] [,3]
x    1    2    3
y    4    5    6

> z[,1] # 選取第一行(column、直)
x y
1 4

> z[1,] # 選取第一列(row、橫)
[1] 1 2 3

> z[1,1:2] # 選取第一列第一到二行
[1] 1 2
```

陣列(array)

基本相關函數

- 陣列加減乘除
- length：計算陣列中的所有元素個數。
- dim：列出維度資訊
- ncol、nrow：計算(column、直) 或 (row、橫) 個數。
- aperm：將陣列轉置

```
> x <- c(1, 2, 3)
> y <- c(4, 5, 6)
```

```
> z = rbind(x, y)
> z
  [,1] [,2] [,3]
x    1    2    3
y    4    5    6
```

```
> length(z)
[1] 6
```

```
> dim(z) # 前者是 row，後者是 column。
```

```
[1] 2 3
```

```
> ncol(z)
```

```
[1] 3
```

```
> nrow(z)
```

```
[1] 2
```

```
> aperm(z) # 等同是從 rbind 轉成 cbind
```

```
  x y
[1,] 1 4
[2,] 2 5
[3,] 3 6
```

```
> z + z
  [,1] [,2] [,3]
x     2     4     6
y     8    10    12
```

```
> z - 2*z
  [,1] [,2] [,3]
x    -1    -2    -3
y    -4    -5    -6
```

```
> z * z # 相對應的陣列元素相乘
  [,1] [,2] [,3]
x     1     4     9
y    16    25    36
```

```
> z / z # 相對應的陣列元素相除
  [,1] [,2] [,3]
x     1     1     1
y     1     1     1
```

矩陣

利用 `matrix` 建立矩陣

當陣列是 2 維的狀況就是所謂的矩陣，可以利用 `matrix` 產生矩陣，也可以用之前產生陣列的方法實作。

```
> matrix(c(1:4), nrow = 2, ncol = 2) # 預設是按照 column 填入資料
```

```
      [,1] [,2]
```

```
[1,]     1     3
```

```
[2,]     2     4
```

```
> matrix(c(1:4), nrow = 2, ncol = 2, byrow = TRUE) # 可以更改成按照 row 填入資料
```

```
      [,1] [,2]
```

```
[1,]     1     2
```

```
[2,]     3     4
```

矩陣

透過指標提取資料

矩陣跟陣列一樣，還是可以透過指標選取矩陣的部份元素。

```
> x <- c(1, 2, 3)
> y <- c(4, 5, 6)

> z = rbind(x, y)
> z
  [,1] [,2] [,3]
x    1    2    3
y    4    5    6

> z[,1] # 選取第一行(column、直)
x y
1 4

> z[1,] # 選取第一列(row、橫)
[1] 1 2 3

> z[1,1:2] # 選取第一列第一到二行
[1] 1 2
```

矩陣

基本相關函數

- `t(x)`：將矩陣轉置。
- `%*%`：矩陣相乘。
- `diag`：產生一個對角矩陣，或回傳矩陣的對角線向量
- `det`：計算矩陣行列式值，一定是要對稱矩陣。
- `solve`：傳回矩陣的反矩陣，非常適合解線性方程式。
- `eigen`：計算矩陣的特徵向量與特徵值。
- `rownames`：修改或查詢 row 名稱。
- `colnames`：修改或查詢 column 名稱。

```
> x <- c(1, 2, 3)
> y <- c(4, 5, 6)
```

```
> z <- rbind(x, y)
```

	[,1]	[,2]	[,3]
x	1	2	3
y	4	5	6

```
> t(z)
```

	x	y
[1,]	1	4
[2,]	2	5
[3,]	3	6

```
> rownames(z) # 還沒修改的時候，是 x 與 y，因為當初是利用兩個向量，利用
rbind 組成，所以會利用向量的變數稱名當作 row 名稱。
```

```
[1] "x" "y"
```

```
> rownames(z) <- c("第一行", "第二行")
```

```
> rownames(z)
```

```
[1] "第一行" "第二行"
```

```
> colnames(z)
```

```
NULL
```

```
> colnames(z) <- c("第一列", "第二列", "第三列")
```

```
> colnames(z)
```

```
[1] "第一列" "第二列" "第三列"
```

```
> z
```

	第一列	第二列	第三列
第一行	1	2	3
第二行	4	5	6


```
>x<-matrix(c(1, 2, 3, 4, 5, 6), 2, 3)
>mdat <- matrix(c(1,2,3, 11,12,13), nrow = 2, ncol=3,
  byrow=TRUE, dimnames = list(c("row1", "row2"), c("C.1",
    "C.2", "C.3")))
>m<-cbind(1:3, 11:13)
>n<-rbind(5:1, 100:104)
>m[2,3]
>n[4,]

>x<-1:20
>dim(x)<-c(4,5) ## convert a vector to a 4X5 array
>x[c(2,4),] ## print the second and the fourth row
>x<-array(1:20, dim=c(4,5))
>y<-t(x)    ## transpose of x
>z<-1:4; k<-z%*%x    ## vector/matrix multiply matrix
```

Common functions

return the objects in the specified environment

```
>ls()
```

remove objects from a specified environment

```
>rm(...)
```

```
>rm(list=ls())
```

give the 'TRUE' indices of a logical object, allowing for array indices

```
> a <- c(1,2,3,4,5)
```

```
> which(a>3)
```

Differences B/W sort(), order(), rank()

```
> aa <- c(94, 86, 18, 79, 28, 68, 7, 27, 84, 10)
```

```
> sort(aa) # 7 10 18 27 28 68 79 84 86 94
```

```
> order(aa) # 7 10 3 8 5 6 4 9 2 1
```

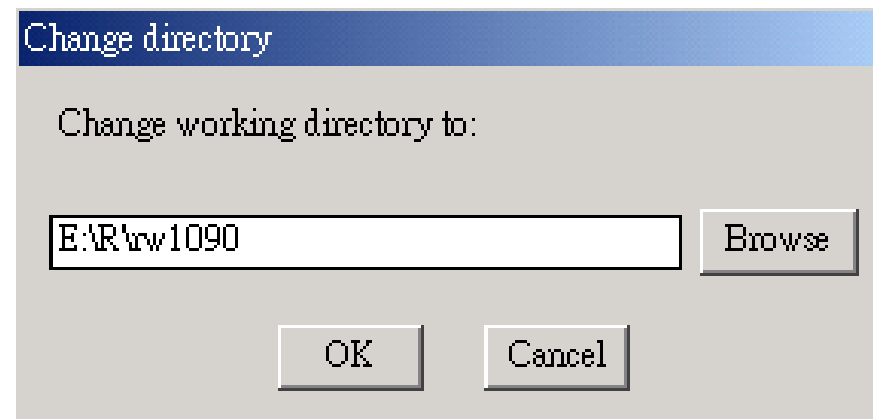
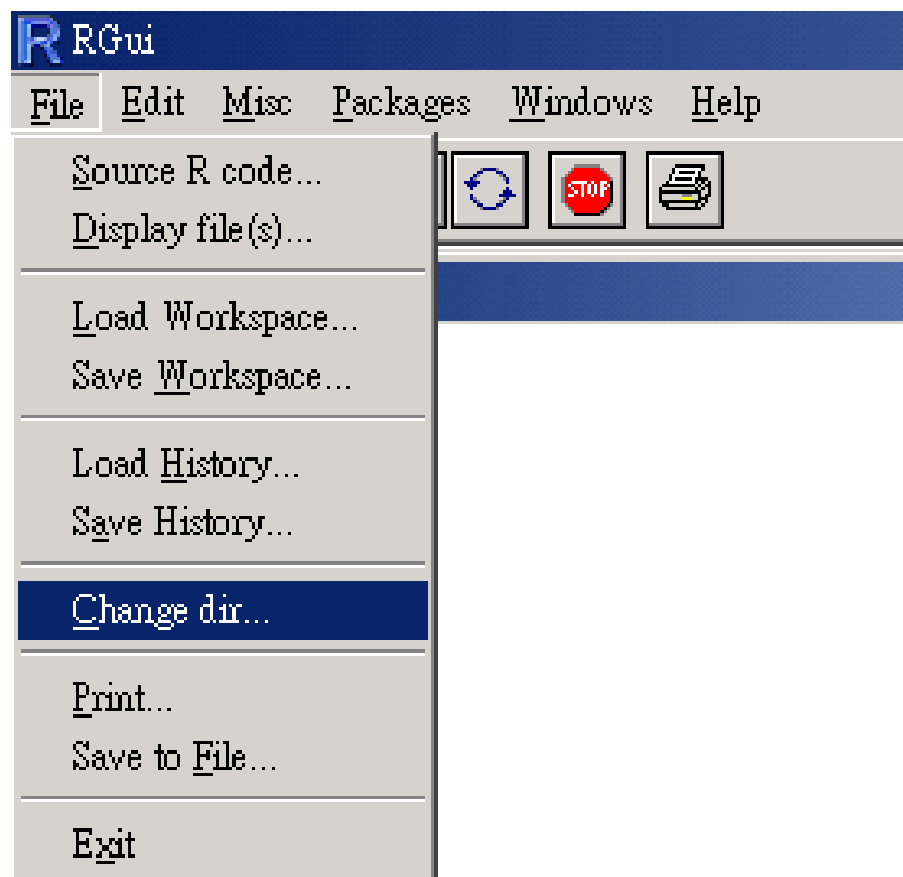
```
> rank(aa) # 10 9 3 7 5 6 1 4 8 2
```

Remove x% data

```
> bb <- which(aa <= quantile(aa,0.3))
```

Setting Working Directory

- >setwd("d:/mydir") # function of setting working directory
- >setwd("d:\\mydir")
- >getwd() # function of getting working directory



Command list

- <https://www.personality-project.org/r/r.commands.html>
- <ftp://cran.r-project.org/pub/R/doc/contrib/Short-refcard.pdf>
- [http://www3.nccu.edu.tw/~99354011/R%20commands\(11.09.13\).pdf](http://www3.nccu.edu.tw/~99354011/R%20commands(11.09.13).pdf)

教學網站

- <http://datascienceandr.org/>

R 語言翻轉教室
版本號：v1.1

- <https://zh-tw.coursera.org/specializations/jhu-data-science>

coursera

HW1

```
> aa <- read.delim("SRBCT_train.txt")
```

```
> bb <- as.matrix(aa[,-c(1,2)])
```

- Get mean & sd value for each gene cross samples
- Get mean & sd value for each sample cross genes
- Select top 30% genes with larger sd values